

Язык программирования С

Лекция 5: стандартная библиотека, ввод-вывод

Лисица Никита

2026

- Набор полезной функциональности, поставляемый вместе с компилятором языка C:
 - `stdio.h` – ввод/вывод (файлы, клавиатура, экран)
 - `stdlib.h` – выделение памяти, различные алгоритмы
 - `string.h` – работа со строками, массивами
 - `math.h` – математические функции
 - `time.h` – работа со временем

- Набор полезной функциональности, поставляемый вместе с компилятором языка C:
 - `stdio.h` – ввод/вывод (файлы, клавиатура, экран)
 - `stdlib.h` – выделение памяти, различные алгоритмы
 - `string.h` – работа со строками, массивами
 - `math.h` – математические функции
 - `time.h` – работа со временем
- Код стандартной библиотеки десятки лет тестировался и улучшался $\implies$ лучше использовать её функции, а не писать свои (разве что в образовательных целях)

- Сам бинарный файл стандартной библиотеки называется по-разному в разных системах:
 - Linux: `libc.so`
 - macOS: `libSystem.B.dylib`
 - Windows: `ucrtbase.dll`

- Сам бинарный файл стандартной библиотеки называется по-разному в разных системах:
 - Linux: `libc.so`
 - macOS: `libSystem.B.dylib`
 - Windows: `ucrtbase.dll`
- Компилятор сам это знает, нам его подставлять руками не нужно :)

- Память, клавиатура, экран, и т.п. – отдельные от CPU устройства

О работе с устройствами

- Память, клавиатура, экран, и т.п. – отдельные от CPU устройства
- Операционная система не даёт к ним прямого доступа, вместо этого мы можем использовать *системные вызовы* с помощью особой инструкции процессора (`syscall` на x86_64)

О работе с устройствами

- Память, клавиатура, экран, и т.п. – отдельные от CPU устройства
- Операционная система не даёт к ним прямого доступа, вместо этого мы можем использовать *системные вызовы* с помощью особой инструкции процессора (`syscall` на x86_64)
- Делать системные вызовы руками очень неудобно (для этого нужно писать на языке ассемблера), и стандартная библиотека оборачивает их в более удобный для программирования интерфейс

О работе с устройствами

- Память, клавиатура, экран, и т.п. – отдельные от CPU устройства
- Операционная система не даёт к ним прямого доступа, вместо этого мы можем использовать *системные вызовы* с помощью особой инструкции процессора (`syscall` на x86_64)
- Делать системные вызовы руками очень неудобно (для этого нужно писать на языке ассемблера), и стандартная библиотека оборачивает их в более удобный для программирования интерфейс
- Например, чтение файла может выполняться так:

О работе с устройствами

- Память, клавиатура, экран, и т.п. – отдельные от CPU устройства
- Операционная система не даёт к ним прямого доступа, вместо этого мы можем использовать *системные вызовы* с помощью особой инструкции процессора (`syscall` на x86_64)
- Делать системные вызовы руками очень неудобно (для этого нужно писать на языке ассемблера), и стандартная библиотека оборачивает их в более удобный для программирования интерфейс
- Например, чтение файла может выполняться так:
 - Программа на C: `fread(...)`

О работе с устройствами

- Память, клавиатура, экран, и т.п. – отдельные от CPU устройства
- Операционная система не даёт к ним прямого доступа, вместо этого мы можем использовать *системные вызовы* с помощью особой инструкции процессора (`syscall` на x86_64)
- Делать системные вызовы руками очень неудобно (для этого нужно писать на языке ассемблера), и стандартная библиотека оборачивает их в более удобный для программирования интерфейс
- Например, чтение файла может выполняться так:
 - Программа на C: `fread(...)`
 - → Стандартная библиотека: `read syscall`

О работе с устройствами

- Память, клавиатура, экран, и т.п. – отдельные от CPU устройства
- Операционная система не даёт к ним прямого доступа, вместо этого мы можем использовать *системные вызовы* с помощью особой инструкции процессора (`syscall` на x86_64)
- Делать системные вызовы руками очень неудобно (для этого нужно писать на языке ассемблера), и стандартная библиотека оборачивает их в более удобный для программирования интерфейс
- Например, чтение файла может выполняться так:
 - Программа на C: `fread(...)`
 - → Стандартная библиотека: `read syscall`
 - → Операционная система: обращается напрямую к устройству

- Стандартная библиотека C работает с файлами как с указателями на специальные структуры **FILE***

- Стандартная библиотека C работает с файлами как с указателями на специальные структуры **FILE***
- **struct** FILE внутри хранит

- Стандартная библиотека C работает с файлами как с указателями на специальные структуры **FILE***
- **struct FILE** внутри хранит
 - *Файловый дескриптор* – число, идентифицирующее открытый файл в ОС

- Стандартная библиотека C работает с файлами как с указателями на специальные структуры **FILE***
- **struct FILE** внутри хранит
 - *Файловый дескриптор* – число, идентифицирующее открытый файл в ОС
 - Временный буфер, использующийся для чтения/записи (чтобы не читать/писать по одному байту)

- Стандартная библиотека C работает с файлами как с указателями на специальные структуры **FILE***
- **struct FILE** внутри хранит
 - *Файловый дескриптор* – число, идентифицирующее открытый файл в ОС
 - Временный буфер, использующийся для чтения/записи (чтобы не читать/писать по одному байту)
 - Текущее состояние – положение в файле, состояние ошибки, флаг конца файла

- Стандартная библиотека C работает с файлами как с указателями на специальные структуры **FILE***
- **struct FILE** внутри хранит
 - *Файловый дескриптор* – число, идентифицирующее открытый файл в ОС
 - Временный буфер, использующийся для чтения/записи (чтобы не читать/писать по одному байту)
 - Текущее состояние – положение в файле, состояние ошибки, флаг конца файла
- Напрямую доступа к этим полям нет, нужно использовать соответствующие функции из **stdio.h**

```
// Открыть файл на диске
FILE *f1 = fopen("in.txt", ...);

// Стандартный ввод: то, что мы вводим с клавиатуры
FILE *f2 = stdin;

// Стандартный вывод: то, что печатается в терминале
FILE *f3 = stdout;
```

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число **100** представляется тремя байтами '1', '0', '0'

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число 100 представляется тремя байтами '1', '0', '0'
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число 100 представляется тремя байтами '1', '0', '0'
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк
 - + Проще читать человеку

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число **100** представляется тремя байтами '1', '0', '0'
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк
 - + Проще читать человеку
 - - Сложнее читать программе, больше размер файла

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число 100 представляется тремя байтами '1', '0', '0'
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк
 - + Проще читать человеку
 - - Сложнее читать программе, больше размер файла
- Бинарные форматы:

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число 100 представляется тремя байтами '1', '0', '0'
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк
 - + Проще читать человеку
 - - Сложнее читать программе, больше размер файла
- Бинарные форматы:
 - Строго следуют определённому порядку байт

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число **100** представляется тремя байтами **'1'**, **'0'**, **'0'**
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк
 - + Проще читать человеку
 - - Сложнее читать программе, больше размер файла
- Бинарные форматы:
 - Строго следуют определённому порядку байт
 - Каждому байту задана его интерпретация, например число **100** может представляться как один байт **0x64**

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число **100** представляется тремя байтами **'1'**, **'0'**, **'0'**
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк
 - + Проще читать человеку
 - - Сложнее читать программе, больше размер файла
- Бинарные форматы:
 - Строго следуют определённому порядку байт
 - Каждому байту задана его интерпретация, например число **100** может представляться как один байт **0x64**
 - + Проще читать программе, компактный размер файла

Текстовые и бинарные форматы

- На физическом диске хранятся просто байты, их интерпретация – вопрос конкретной программы и конкретного формата файла
- Текстовые форматы:
 - Интерпретируются, как последовательность текстовых символов: например, число **100** представляется тремя байтами **'1'**, **'0'**, **'0'**
 - Часто содержит спецсимволы: пробелы, табуляцию, перевод строк
 - + Проще читать человеку
 - - Сложнее читать программе, больше размер файла
- Бинарные форматы:
 - Строго следуют определённому порядку байт
 - Каждому байту задана его интерпретация, например число **100** может представляться как один байт **0x64**
 - + Проще читать программе, компактный размер файла
 - - Сложно читать человеку

```
FILE* fopen(char *path, char *mode);
```

```
FILE* fopen(char *path, char *mode);
```

- path – путь до файла

```
FILE* fopen(char *path, char *mode);
```

- `path` – путь до файла
- `mode` – режим открытия файла, может состоять из нескольких СИМВОЛОВ:

```
FILE* fopen(char *path, char *mode);
```

- `path` – путь до файла
- `mode` – режим открытия файла, может состоять из нескольких символов:
 - Один из `'r'/'w'/'a'` – открыть на чтение, на перезапись, или на дозапись в конец (при записи файл создаётся, если не существовал)

```
FILE* fopen(char *path, char *mode);
```

- `path` – путь до файла
- `mode` – режим открытия файла, может состоять из нескольких символов:
 - Один из `'r'/'w'/'a'` – открыть на чтение, на перезапись, или на дозапись в конец (при записи файл создаётся, если не существовал)
 - `'+'` – режим “обновления” файла: можно и читать, и писать

```
FILE* fopen(char *path, char *mode);
```

- `path` – путь до файла
- `mode` – режим открытия файла, может состоять из нескольких символов:
 - Один из `'r'/'w'/'a'` – открыть на чтение, на перезапись, или на дозапись в конец (при записи файл создаётся, если не существовал)
 - `'+'` – режим “обновления” файла: можно и читать, и писать
 - `'x'` – выдать ошибку, если файл открыт на запись (`'w'/'a'`) но не существует

```
FILE* fopen(char *path, char *mode);
```

- `path` – путь до файла
- `mode` – режим открытия файла, может состоять из нескольких символов:
 - Один из `'r'/'w'/'a'` – открыть на чтение, на перезапись, или на дозапись в конец (при записи файл создаётся, если не существовал)
 - `'+'` – режим “обновления” файла: можно и читать, и писать
 - `'x'` – выдать ошибку, если файл открыт на запись (`'w'/'a'`) но не существует
 - `'b'` – открыть файл в бинарном режиме (по умолчанию режим текстовый)

```
FILE* fopen(char *path, char *mode);
```

- `path` – путь до файла
- `mode` – режим открытия файла, может состоять из нескольких символов:
 - Один из `'r'/'w'/'a'` – открыть на чтение, на перезапись, или на дозапись в конец (при записи файл создаётся, если не существовал)
 - `'+'` – режим “обновления” файла: можно и читать, и писать
 - `'x'` – выдать ошибку, если файл открыт на запись (`'w'/'a'`) но не существует
 - `'b'` – открыть файл в бинарном режиме (по умолчанию режим текстовый)
 - `'t'` – **нестандартное расширение**: открыть файл в текстовом режиме

- В бинарном режиме файл читается/пишется, как есть, без преобразований

Текстовый и бинарный режимы

- В бинарном режиме файл читается/пишется, как есть, без преобразований
- В текстовом режиме под Windows:

Текстовый и бинарный режимы

- В бинарном режиме файл читается/пишется, как есть, без преобразований
- В текстовом режиме под Windows:
 - При чтении перевод строки "`\r\n`" превращается в "`\n`", при записи – наоборот

Текстовый и бинарный режимы

- В бинарном режиме файл читается/пишется, как есть, без преобразований
- В текстовом режиме под Windows:
 - При чтении перевод строки "**\r\n**" превращается в "**\n**", при записи – наоборот
 - Особое значение байта **0x1a** означает конец файла

Текстовый и бинарный режимы

- В бинарном режиме файл читается/пишется, как есть, без преобразований
- В текстовом режиме под Windows:
 - При чтении перевод строки `"\r\n"` превращается в `"\n"`, при записи – наоборот
 - Особое значение байта `0x1a` означает конец файла
- В текстовом режиме под другими платформами:

Текстовый и бинарный режимы

- В бинарном режиме файл читается/пишется, как есть, без преобразований
- В текстовом режиме под Windows:
 - При чтении перевод строки `"\r\n"` превращается в `"\n"`, при записи – наоборот
 - Особое значение байта `0x1a` означает конец файла
- В текстовом режиме под другими платформами:
 - Ничего особого не происходит :)

Текстовый и бинарный режимы

- В бинарном режиме файл читается/пишется, как есть, без преобразований
- В текстовом режиме под Windows:
 - При чтении перевод строки `"\r\n"` превращается в `"\n"`, при записи – наоборот
 - Особое значение байта `0x1a` означает конец файла
- В текстовом режиме под другими платформами:
 - Ничего особого не происходит :)
- Не важно при работе с текстовыми форматами

Текстовый и бинарный режимы

- В бинарном режиме файл читается/пишется, как есть, без преобразований
- В текстовом режиме под Windows:
 - При чтении перевод строки `"\r\n"` превращается в `"\n"`, при записи – наоборот
 - Особое значение байта `0x1a` означает конец файла
- В текстовом режиме под другими платформами:
 - Ничего особого не происходит :)
- Не важно при работе с текстовыми форматами
- При работе с бинарными форматами **обязательно** нужно указывать флаг `"b"`

```
FILE *file = fopen("my_file.txt", "r");
if (file == NULL) {
    // Обрабатываем ошибку открытия файла
}

// Работаем с файлом...

// Закрываем файл
fclose(file);
```

```
FILE *file = fopen("my_file.txt", "r");  
if (file == NULL) {  
    // Обрабатываем ошибку открытия файла  
}  
  
// Работаем с файлом...  
  
// Закрываем файл  
fclose(file);
```

- Зачем закрывать файл, если при закрытии программы ОС его закроет сама?

```
FILE *file = fopen("my_file.txt", "r");  
if (file == NULL) {  
    // Обрабатываем ошибку открытия файла  
}  
  
// Работаем с файлом...  
  
// Закрываем файл  
fclose(file);
```

- Зачем закрывать файл, если при закрытии программы ОС его закроет сама?
- На **struct FILE** тратится память

```
FILE *file = fopen("my_file.txt", "r");
if (file == NULL) {
    // Обрабатываем ошибку открытия файла
}

// Работаем с файлом...

// Закрываем файл
fclose(file);
```

- Зачем закрывать файл, если при закрытии программы ОС его закроет сама?
- На **struct FILE** тратится память
- Число файловых дескрипторов на один процесс ограничено

Работа с файлами

```
FILE *file = fopen("my_file.txt", "r");  
if (file == NULL) {  
    // Обрабатываем ошибку открытия файла  
}  
  
// Работаем с файлом...  
  
// Закрываем файл  
fclose(file);
```

- Зачем закрывать файл, если при закрытии программы ОС его закроет сама?
- На **struct FILE** тратится память
- Число файловых дескрипторов на один процесс ограничено
- Открытый файл нельзя удалить

Бинарные файлы

```
// Прочитать count объектов, каждый по size байт,  
// из файла в буфер dst_buf  
size_t fread(void *dst_buf, size_t size, size_t count, FILE *file);  
  
// Записать count объектов, каждый по size байт,  
// из буфера src_buf в файл  
size_t fwrite(void *src_buf, size_t size, size_t count, FILE *file);
```

Бинарные файлы

```
// Прочитать count объектов, каждый по size байт,  
// из файла в буфер dst_buf  
size_t fread(void *dst_buf, size_t size, size_t count, FILE *file);  
  
// Записать count объектов, каждый по size байт,  
// из буфера src_buf в файл  
size_t fwrite(void *src_buf, size_t size, size_t count, FILE *file);
```

- Почему не просто количество байт?

Бинарные файлы

```
// Прочитать count объектов, каждый по size байт,  
// из файла в буфер dst_buf  
size_t fread(void *dst_buf, size_t size, size_t count, FILE *file);  
  
// Записать count объектов, каждый по size байт,  
// из буфера src_buf в файл  
size_t fwrite(void *src_buf, size_t size, size_t count, FILE *file);
```

- Почему не просто количество байт?
- Функции возвращают количество прочитанных/записанных объектов

Бинарные файлы

```
// Открыли на чтение в бинарном режиме
FILE *file1 = fopen("in.bin", "rb");

// Открыли на запись в бинарном режиме
FILE *file2 = fopen("out.bin", "wb");

int array[100];

// Прочитали 100 int'ов из файла в массив
fread(array, sizeof(int), 100, file1);

// Записали 100 int'ов из массива в файл
fwrite(array, sizeof(int), 100, file2);
```

Текстовые файлы

```
// Открыли на чтение в текстовом режиме
FILE *file1 = fopen("in.text", "r");

// Открыли на запись в текстовом режиме
FILE *file2 = fopen("out.txt", "w");

int i = 0;
float f = 0.f;

// Прочитать целое число и число с плавающей
// точкой из файла в текстовом формате
fscanf(file1, "%i %f", &i, &f);

// Записать их в другой файл, тоже в
// текстовом формате
fprintf(file2, "%i %f", i, f);
```

Текстовые файлы

```
// Открыли на чтение в текстовом режиме
FILE *file1 = fopen("in.text", "r");

// Открыли на запись в текстовом режиме
FILE *file2 = fopen("out.txt", "w");

int i = 0;
float f = 0.f;

// Прочитать целое число и число с плавающей
// точкой из файла в текстовом формате
fscanf(file1, "%i %f", &i, &f);

// Записать их в другой файл, тоже в
// текстовом формате
fprintf(file2, "%i %f", i, f);
```

- Также возвращают количество прочитанных/записанных объектов

```
char string[256];  
fscanf(file, "%s", str);
```

- Прочитает всё до первого *whitespace* символа:
пробела/табуляции/перевода строки

```
char string[256];  
fscanf(file, "%s", str);
```

- Прочитает всё до первого *whitespace* символа: пробела/табуляции/перевода строки
- В чём проблема такого кода?

```
char string[256];  
fscanf(file, "%s", str);
```

- Прочитает всё до первого *whitespace* символа: пробела/табуляции/перевода строки
- В чём проблема такого кода? Мы не знаем заранее размер строки, и `fscanf` не знает размер нашего буфера

```
char string[256];  
fscanf(file, "%s", str);
```

- Прочитает всё до первого *whitespace* символа: пробела/табуляции/перевода строки
- В чём проблема такого кода? Мы не знаем заранее размер строки, и `fscanf` не знает размер нашего буфера
- Альтернативы:

```
char string[256];  
fscanf(file, "%s", str);
```

- Прочитает всё до первого *whitespace* символа: пробела/табуляции/перевода строки
- В чём проблема такого кода? Мы не знаем заранее размер строки, и `fscanf` не знает размер нашего буфера
- Альтернативы:
 - `fgets(string, 255, file)`

```
char string[256];  
fscanf(file, "%s", str);
```

- Прочитает всё до первого *whitespace* символа: пробела/табуляции/перевода строки
- В чём проблема такого кода? Мы не знаем заранее размер строки, и `fscanf` не знает размер нашего буфера
- Альтернативы:
 - `fgets(string, 255, file)`
 - `fscanf(file, "%255s", string)`

```
char string[256];  
fscanf(file, "%s", str);
```

- Прочитает всё до первого *whitespace* символа: пробела/табуляции/перевода строки
- В чём проблема такого кода? Мы не знаем заранее размер строки, и `fscanf` не знает размер нашего буфера
- Альтернативы:
 - `fgets(string, 255, file)`
 - `fscanf(file, "%255s", string)`
 - `fscanf_s(file, "%s", string, 255)`

- Функции `printf`/`scanf` работают со *стандартными потоками* ввода/вывода (`stdin`/`stdout`)

- Функции `printf/scanf` работают со *стандартными потоками* ввода/вывода (`stdin/stdout`)
- Функции `fprintf/fscanf` работают с произвольными файлами

printf/fprintf/sprintf

- Функции `printf/scanf` работают со *стандартными потоками* ввода/вывода (`stdin/stdout`)
- Функции `fprintf/fscanf` работают с произвольными файлами
- Функции `sprintf/sscanf` работают со строками

printf/fprintf/sprintf

- Функции `printf/scanf` работают со *стандартными потоками* ввода/вывода (`stdin/stdout`)
- Функции `fprintf/fscanf` работают с произвольными файлами
- Функции `sprintf/sscanf` работают со строками

```
int a, b;  
char str1[] = "3 4";  
sscanf(str1, "%d %d", &a, &b);  
  
char str2[256];  
sprintf(str2, "%d + %d = %d", a, b, c);
```

Конец файла, ошибки

```
while (true) {  
    fread(...);  
  
    if (feof(file)) {  
        // Конец файла  
        break;  
    }  
  
    if (ferror(file)) {  
        // Обрабатываем ошибку  
    }  
  
    ...  
}
```

Конец файла, ошибки

```
while (true) {  
    fread(...);  
  
    if (feof(file)) {  
        // Конец файла  
        break;  
    }  
  
    if (ferror(file)) {  
        // Обрабатываем ошибку  
    }  
  
    ...  
}
```

- `feof(file)` возвращает состояние флага EOF (End Of File)

Конец файла, ошибки

```
while (true) {  
    fread(...);  
  
    if (feof(file)) {  
        // Конец файла  
        break;  
    }  
  
    if (ferror(file)) {  
        // Обработка ошибки  
    }  
  
    ...  
}
```

- `feof(file)` возвращает состояние флага EOF (End Of File)
- `ferror(file)` возвращает состояние ошибок (узнать саму ошибку можно через `errno`, про это поговорим позже)

Конец файла, ошибки

```
while (true) {  
    fread(...);  
  
    if (feof(file)) {  
        // Конец файла  
        break;  
    }  
  
    if (ferror(file)) {  
        // Обработаем ошибку  
    }  
  
    ...  
}
```

- `feof(file)` возвращает состояние флага EOF (End Of File)
- `ferror(file)` возвращает состояние ошибок (узнать саму ошибку можно через `errno`, про это поговорим позже)
- Флаги выставляются **при попытке чтения**: сначала мы попробуем прочитать за концом файла, и только потом выставится флаг EOF

Буферизация, flush

```
int main() {  
    FILE *file = fopen("file.txt", "w");  
    fprintf(file, "%i", 42);  
  
    // Произошла какая-то ошибка  
    if (some_error) {  
        return 0;  
    }  
  
    // Нормальное завершение  
    fclose(file);  
    return 0;  
}
```

- В чём проблема кода?

Буферизация, flush

```
int main() {  
    FILE *file = fopen("file.txt", "w");  
    fprintf(file, "%i", 42);  
  
    // Произошла какая-то ошибка  
    if (some_error) {  
        return 0;  
    }  
  
    // Нормальное завершение  
    fclose(file);  
    return 0;  
}
```

- В чём проблема кода? Записанные данные лежат в промежуточном буфере в **struct** FILE, и могут не попасть на диск

Буферизация, flush

```
int main() {  
    FILE *file = fopen("file.txt", "w");  
    fprintf(file, "%i", 42);  
  
    // Произошла какая-то ошибка  
    if (some_error) {  
        return 0;  
    }  
  
    // Нормальное завершение  
    fclose(file);  
    return 0;  
}
```

- В чём проблема кода? Записанные данные лежат в промежуточном буфере в `struct FILE`, и могут не попасть на диск
- Можно явно вызвать `fflush(file)` – это форсирует сброс данных на диск